

TravelEase

Designing a Scalable Travel Booking Platform

System Design Final Examination Project

Student Name: Perni Bharath Raghavendra

Roll Number: 150096724139

Programme: B.Tech CSE, Semester 4

Cohort: Mark Zuckerberg Cohort

University: ITM Skills University

Course: System Design

GitHub Repository: github.com/bharath-541/travelease-system-design

Live Application: travelease-system-design.onrender.com

Flask implementation, architecture, database design, algorithms, scalability, and fault tolerance

Student Name: Perni Bharath Raghavendra

Roll Number: 150096724139

Programme: B.Tech CSE, Semester 4

Cohort: Mark Zuckerberg Cohort

University: ITM Skills University

Course: System Design

GitHub Repository: <https://github.com/bharath-541/travelease-system-design>

Live Application: <https://travelease-system-design.onrender.com/>

Problem Statement

TravelEase is an online platform for searching, comparing, and booking flights, hotels, buses, trains, and holiday packages. It must remain responsive while millions of users search concurrently, receive frequent price and inventory updates, and attempt bookings against many independent travel providers.

The design must address dynamic pricing, temporary inventory holds, concurrent bookings, secure payment processing, provider API delays, reliable confirmations, cancellations, refunds, traffic spikes, and partial failures. The implementation in this repository is a working academic simulation of the most important workflows, while the proposed architecture describes how the same boundaries scale in production.

Proposed Solution

TravelEase uses a service-oriented, event-driven architecture. Search is separated from authoritative booking state so that read-heavy traffic can scale independently. Listings are indexed in OpenSearch and cached in Redis, while bookings, inventory, payments, and idempotency records remain in a strongly consistent relational database.

A Booking Manager coordinates the reservation workflow as a Saga. It first creates a short-lived inventory hold in a transaction, then authorizes payment, asks the external provider to revalidate inventory, and finally confirms the booking. If payment or provider confirmation fails, compensating actions release inventory and refund any approved payment.

The included Flask application demonstrates these states using SQLite, provider simulators, and payment simulators. It is intentionally local and deterministic enough for evaluation, but its modules mirror production service boundaries.

Q1. Requirements Analysis

Functional Requirements

- Register and manage user profiles and contact information.
- Search flights, hotels, buses, trains, and packages by origin, destination, date, and type.
- Filter and sort results by price, rating, availability, duration, and provider.
- Compare dynamically calculated prices and view availability warnings.
- Create temporary inventory holds before payment.
- Process payments idempotently and store transaction history.

- Revalidate inventory with external providers before final confirmation.
- Generate booking and provider confirmation references.
- Cancel confirmed bookings, release inventory, and initiate refunds.
- Notify users through email, SMS, or push channels in a production deployment.
- Maintain an auditable sequence of booking, payment, provider, cancellation, and refund events.

Non-Functional Requirements

- **Scalability:** Search traffic can be hundreds of times larger than booking traffic. Stateless services, horizontal scaling, partitioned indexes, caching, and asynchronous processing are therefore required.
- **Availability:** Search should remain available even if one supplier is slow. Multi-zone deployment, health checks, replicas, circuit breakers, and degraded results protect the customer experience.
- **Low latency:** Users compare many options and abandon slow pages. Cached queries and search indexes should target sub-second responses, while provider calls use timeouts and parallel fan-out.
- **Consistency:** Inventory and booking state require strong consistency to prevent overselling. Search results may be eventually consistent, but final booking always revalidates authoritative inventory.
- **Security:** TLS, encrypted sensitive data, tokenized payment methods, least-privilege access, input validation, audit logs, and PCI-DSS-aligned payment boundaries are required.
- **Reliability:** Idempotency keys, retry policies, outbox events, dead-letter queues, and compensating transactions prevent duplicate charges and stranded inventory.
- **Observability:** Metrics, structured logs, distributed traces, business KPIs, and alerting are required for provider latency, payment success, booking conversion, and inconsistency detection.

Importance of Scalability, Consistency, and Low-Latency Search

Scalability lets TravelEase absorb seasonal peaks without making every subsystem equally large. Search nodes and caches can scale aggressively, while the lower-volume transactional booking path scales around database partitions and connection limits.

Consistency is critical at the commit point. A stale search result is inconvenient, but a charged customer without a seat or room is a severe failure. TravelEase therefore accepts eventual consistency for discovery and requires transactional checks for inventory holds, payment state, and final confirmation.

Low-latency search directly affects conversion. Search requests should use denormalized documents, precomputed facets, geographic indexes, cached popular routes, pagination, and parallel supplier calls. Slow suppliers should not block the complete result set.

Q2. System Architecture Design

TravelEase Distributed Architecture

Low-latency search, transactional booking, event-driven synchronization, and failure compensation

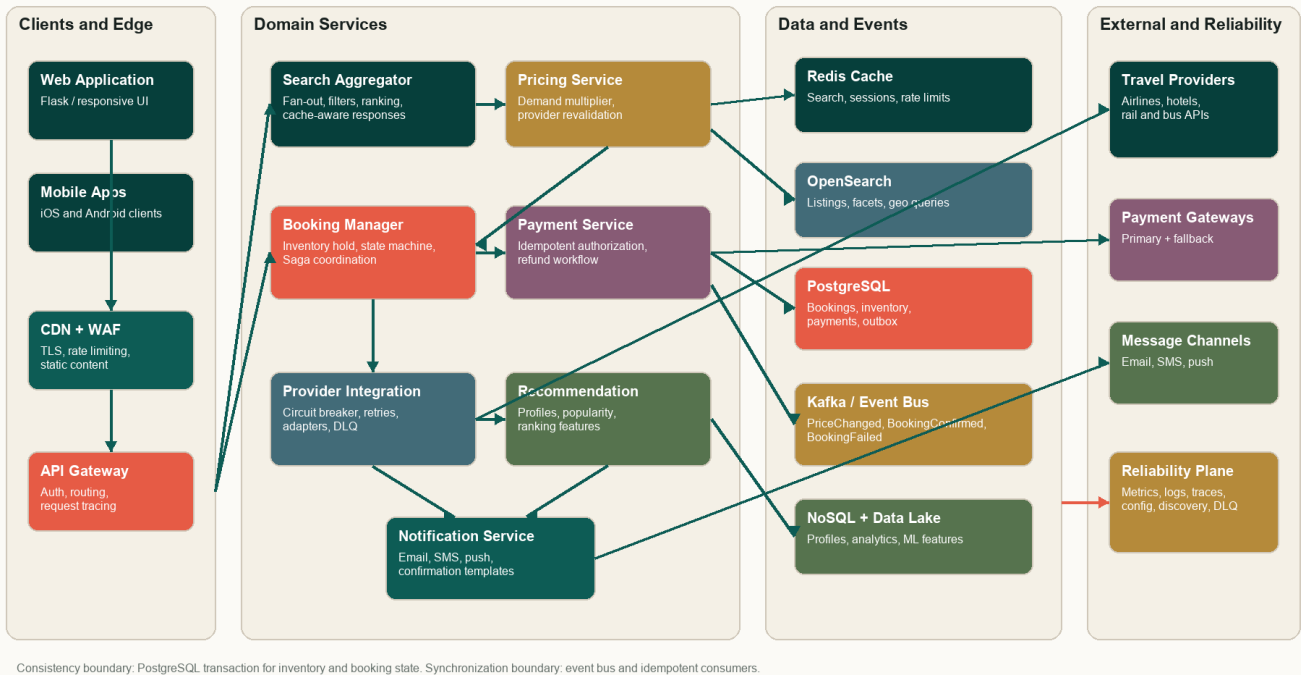


Figure 1. Proposed TravelEase distributed architecture

Component Interaction

- CDN and WAF terminate TLS, cache static assets, block abusive traffic, and protect the public edge.
- API Gateway handles authentication, quotas, routing, request IDs, and API versioning.
- Search Aggregator queries Redis and OpenSearch, merges provider freshness signals, applies filters, and ranks results.
- Pricing Service calculates the displayed quote from base price, demand, scarcity, taxes, fees, and promotions.
- Booking Manager owns the reservation state machine and coordinates payment and provider steps as a Saga.
- Payment Service uses idempotency keys and supports multiple gateways with circuit breakers and fallback routing.
- Provider Integration Service contains airline, hotel, rail, and bus adapters with provider-specific transformations.
- Recommendation Service consumes browsing and booking signals to rank destinations and listings.
- Kafka or another event bus distributes price changes, booking events, notification commands, and analytics events.
- PostgreSQL stores authoritative inventory, booking, payment, and outbox state. Read replicas support reporting.
- OpenSearch stores denormalized searchable listings. Redis caches popular queries and short-lived sessions.
- Observability includes metrics, logs, traces, dashboards, service-level objectives, and reconciliation jobs.

Architectural Justification

Search and booking are separated because they have different consistency and scaling requirements. The search path is read-heavy and tolerant of short-lived staleness. The booking path is write-sensitive and must prevent lost updates and duplicate actions.

The event bus reduces synchronous dependencies. Confirmation, notifications, search-index updates, analytics, and recommendation features can consume events independently. The transactional outbox pattern should publish database changes reliably without a distributed two-phase commit.

Q3. Search and Booking Workflow

Search and Filtering

1. The client sends travel type, route, dates, traveller count, and filters to the API Gateway.
2. The Search Aggregator normalizes the query and checks Redis for a cached result.
3. On a cache miss, it queries OpenSearch and optionally fans out to selected providers in parallel.
4. Results are merged, deduplicated, ranked, filtered, and returned with a short quote expiry.
5. Slow or unavailable providers are skipped after a timeout, and the response identifies any degraded coverage.

Search documents include provider, route, dates, location, amenities, rating, base price range, and availability hints. They are updated asynchronously from provider feeds and booking events.

Dynamic Pricing

Displayed prices are quotes, not permanent values. The pricing calculation can include:

$\text{price} = \text{base_price} * \text{demand_multiplier} * \text{scarcity_multiplier} + \text{taxes} + \text{service_fee}$

The implementation recalculates scarcity from available inventory and total inventory. A production service would also use promotions, customer segment, provider commissions, currency conversion, fare class, and quote-expiry rules.

Secure Booking

1. The user selects a listing.
2. Booking Manager opens a short transaction, locks or conditionally updates inventory, and creates a HELD booking with an expiry time.
3. The user submits payment. Payment Service uses an idempotency key so retries cannot create duplicate charges.
4. After authorization, Provider Integration revalidates inventory and obtains the supplier confirmation reference.
5. Booking Manager marks the booking CONFIRMED and writes an outbox event.
6. Notification Service consumes the event and sends confirmation messages.

Failure and Compensation

- Payment decline: mark PAYMENT_FAILED and return held inventory.
- Provider timeout or rejection after payment: release inventory, mark PROVIDER_FAILED, and issue a compensating refund.
- Hold expiry: mark EXPIRED and return inventory.
- Duplicate request: return the earlier booking or payment result using the idempotency key.
- Event consumer failure: retry with exponential backoff and move poison messages to a dead-letter queue.
- Database/event publication gap: use a transactional outbox and idempotent consumers.

TravelEase Booking Sequence

Idempotent payment, provider revalidation, and Saga compensation

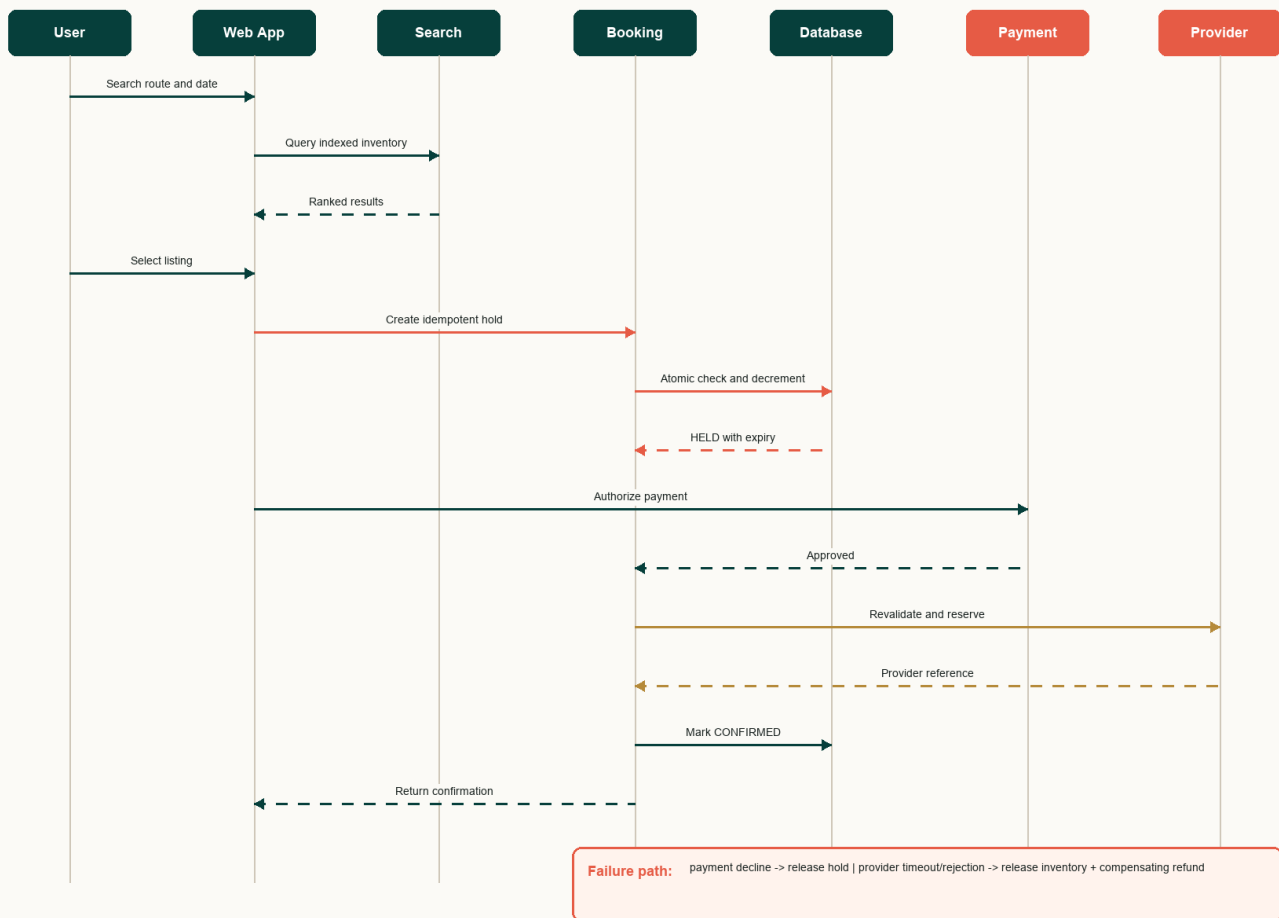


Figure 2. Booking sequence and compensation path

Q4. Database Design

Relational Transactional Store

PostgreSQL is recommended for bookings, inventory, payments, provider confirmations, refunds, and audit history. These records require constraints, transactions, indexes, and well-defined state transitions.

Core tables:

- users: profile and contact details.
- listings: normalized authoritative inventory references and current availability.
- bookings: user, listing, traveller count, status, quoted totals, hold expiry, and idempotency key.
- payments: gateway reference, status, amount, failure reason, and payment idempotency key.
- provider_confirmations: supplier response, reference, status, message, and retry attempt.
- transaction_history: immutable business event audit trail.
- outbox_events in production: unpublished domain events committed in the booking transaction.

Important indexes include listings by travel type, origin, destination, and departure date; bookings by status and creation time; unique idempotency keys; and provider references.

Search and High-Volume Stores

- OpenSearch: denormalized listings, autocomplete, filters, geo queries, sorting, and facets.
- Redis: popular search results, session data, rate limits, quote metadata, and distributed locks where appropriate.
- NoSQL store: user preferences, clickstream-derived profiles, flexible recommendation features, and high-volume session data.
- Object storage/data lake: raw provider feeds, reports, model features, and historical analytics.

Synchronization Strategy

PostgreSQL remains the source of truth for booking state. Change-data-capture or outbox consumers update OpenSearch and caches. Events are versioned and consumers are idempotent. Reconciliation jobs compare internal bookings with provider records and automatically flag mismatches.

Q5. Algorithm and Python Implementation

The project contains a structured Flask implementation:

- app/services.py: search cache, dynamic pricing, inventory hold, payment, provider confirmation, compensation, cancellation.
- app/db.py: SQLite schema, indexes, sample users, and seeded provider inventory.
- app/routes.py: HTTP routes for search, review, payment simulation, confirmation, cancellation, and system evidence.
- app/templates and app/static: responsive user interface and workflow states.

Simplified Booking Algorithm

```
begin_immediate_transaction()
listing = load_listing(listing_id)
assert listing.inventory_available >= travellers
quote = calculate_dynamic_price(listing, travellers)
conditionally_decrement_inventory(listing_id, travellers)
create_booking(status="HELD", hold_expiry=now_plus_10_minutes)
commit()

payment = authorize_with_idempotency_key(booking_id)
if payment.declined:
    release_inventory()
    mark_booking("PAYMENT_FAILED")
elif provider_confirms():
    mark_booking("CONFIRMED")
    publish_booking_confirmed()
else:
    release_inventory()
    refund_payment()
    mark_booking("PROVIDER_FAILED")
```

BEGIN IMMEDIATE in SQLite demonstrates the production idea of a short write transaction. The conditional inventory update ensures only one concurrent request can consume the final unit. In PostgreSQL this could use SELECT FOR UPDATE, an atomic UPDATE with a predicate, or optimistic concurrency with a version column.

Search Optimization

The local implementation caches normalized queries for thirty seconds. Production would use a distributed cache, query result TTLs, cache-key normalization, OpenSearch shards and replicas, autocomplete analyzers, geo indexes, price/date facets, and asynchronous index refresh.

Q6. Scalability and Fault Tolerance

Scaling Strategy

- Run stateless API, search, pricing, and integration services behind load balancers.
- Autoscale on latency, request rate, queue depth, CPU, and provider concurrency.
- Separate read-heavy search infrastructure from transaction-heavy booking infrastructure.
- Partition OpenSearch by market and date where appropriate, and use replicas for availability.
- Partition high-volume booking data by region or booking identifier after a single cluster becomes limiting.
- Use database connection pools, read replicas, and archival policies.
- Cache popular destinations and routes at the edge and Redis layers.
- Process notifications, analytics, recommendation updates, and reconciliation asynchronously.
- Apply backpressure, quotas, and load shedding during spikes.

Payment Gateway Outage

Use circuit breakers, strict timeouts, health-aware gateway routing, and queued retry only when the payment operation is safe and idempotent. Display a pending state rather than repeatedly charging. Reconciliation checks the gateway before a retry if the previous response was ambiguous.

Provider API Delay

Use provider-specific timeouts, bounded retries with jitter, circuit breakers, bulkheads, cached search data, and degraded search results. Booking confirmation should move to a pending state if the provider supports asynchronous status checks. Otherwise, compensate payment and inventory safely.

Booking Inconsistency

Use idempotency keys, booking state transition validation, provider reference uniqueness, outbox events, immutable audit history, and scheduled reconciliation. Manual operations tools should allow support teams to inspect and repair exceptional states without editing raw records.

High Availability

Deploy services across multiple availability zones, use replicated databases, maintain tested backups, and define recovery objectives. Multi-region search can be active-active. Booking writes may use a regional primary with controlled failover to preserve consistency.

Technology Stack

- Frontend: Jinja templates, semantic HTML, responsive CSS, vanilla JavaScript.
- Backend: Python 3 and Flask.
- Local database: SQLite for a portable academic demonstration.
- Production database: PostgreSQL with replicas and partitioning.
- Search: OpenSearch or Elasticsearch.
- Cache: Redis.

- Messaging: Apache Kafka or managed equivalent.
- Documents: ReportLab and Pillow.
- Testing: pytest and Flask test client.
- Observability: OpenTelemetry, Prometheus, Grafana, centralized logs, and alerting.

Module Description

- SearchService: normalized filters, sorting, cache simulation, and search event persistence.
- PricingService: demand, scarcity, taxes, fees, and quote totals.
- BookingService: inventory transaction, temporary hold, expiry, and booking retrieval.
- PaymentService: idempotent authorization, decline handling, provider coordination, and compensation.
- ProviderIntegrationSimulator: success, timeout, and rejection responses.
- CancellationService: allowed-state validation, inventory restoration, and refund state.
- Routes: presentation-layer orchestration with clear redirects and failure pages.
- Report and diagram scripts: reproducible academic artifacts.

Implementation Screenshots

The following screenshots are generated during browser verification:

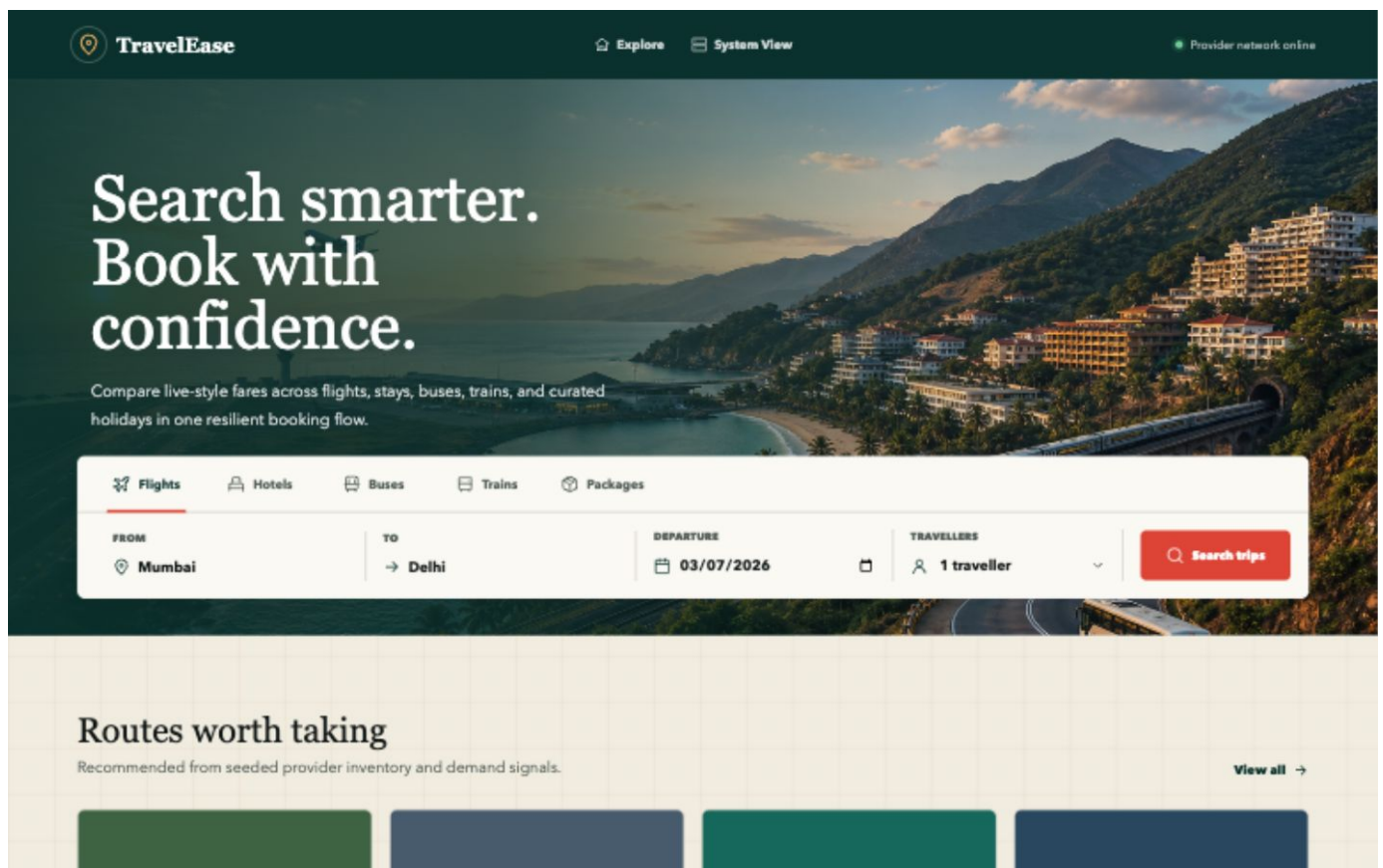


Figure 3. TravelEase search experience

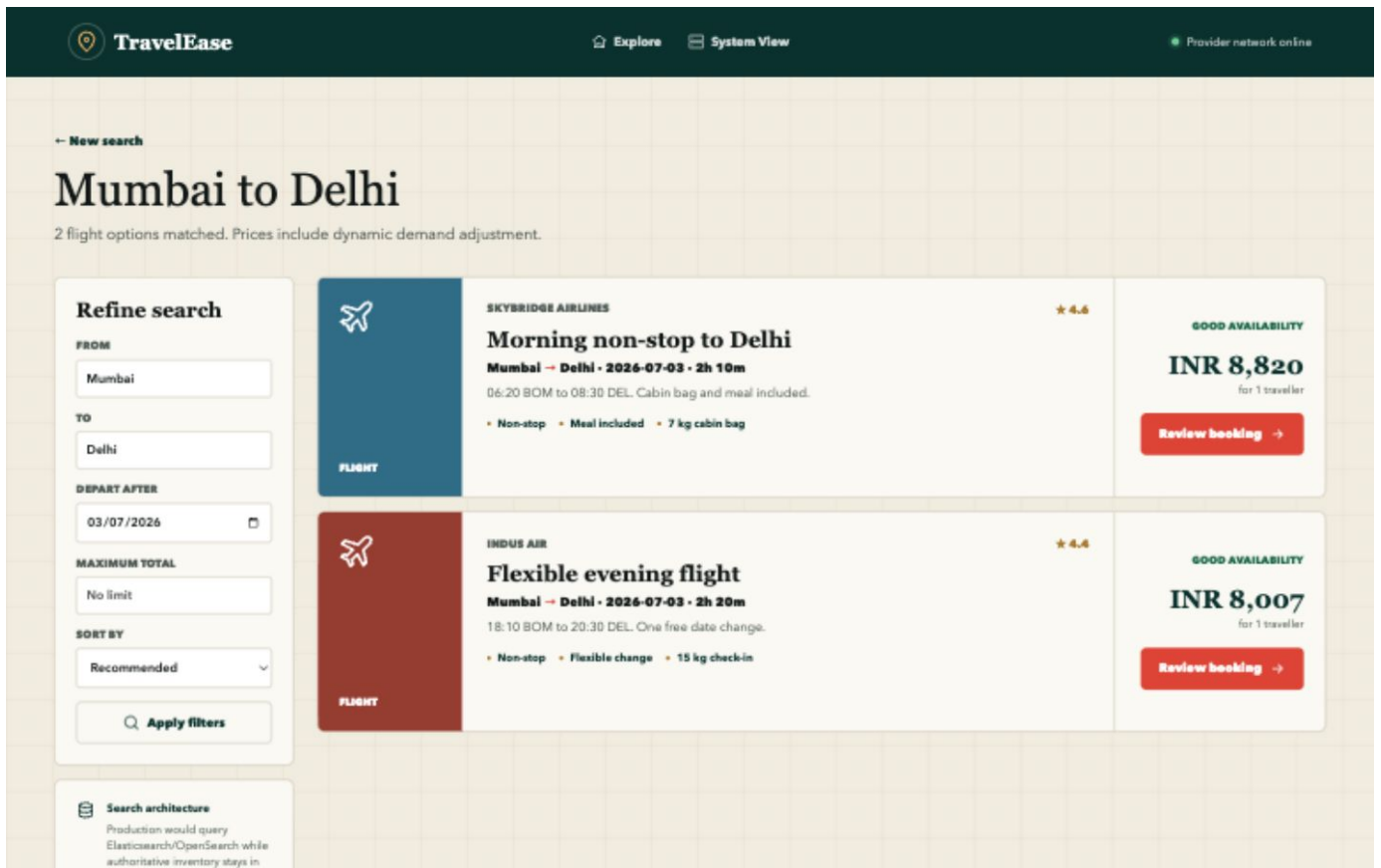


Figure 4. Search results with dynamic pricing and availability

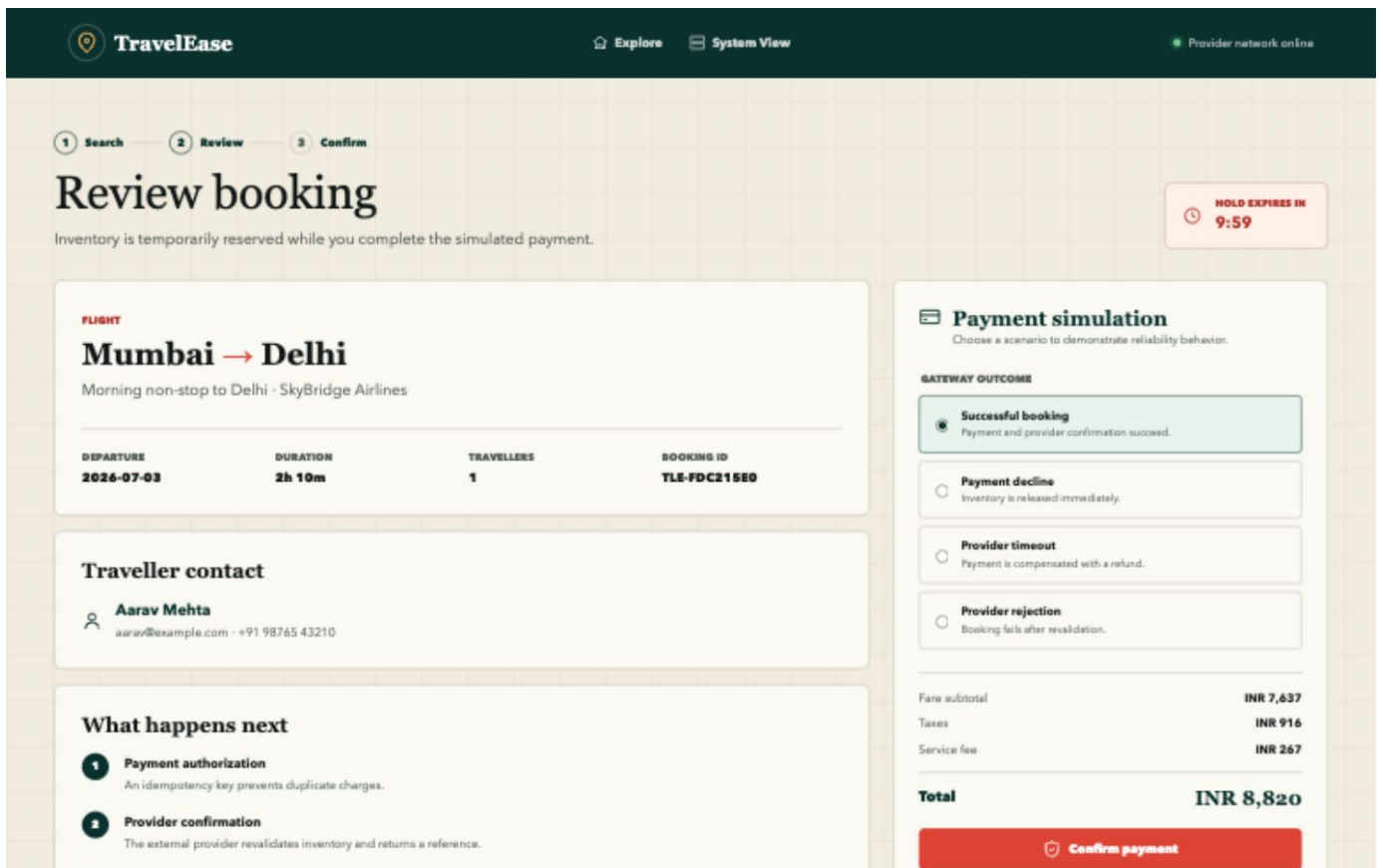


Figure 5. Inventory hold and payment simulation

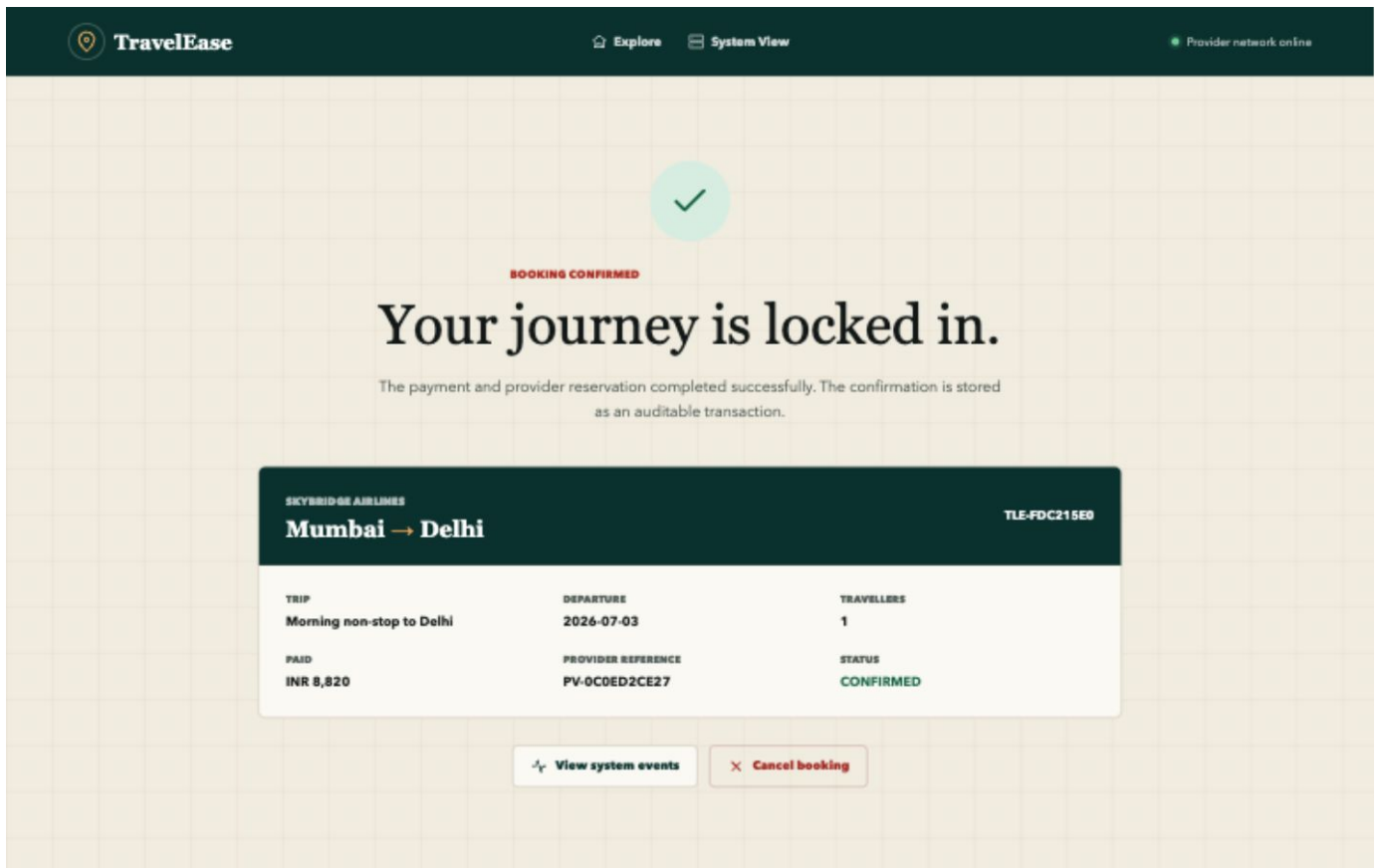


Figure 6. Provider-backed booking confirmation

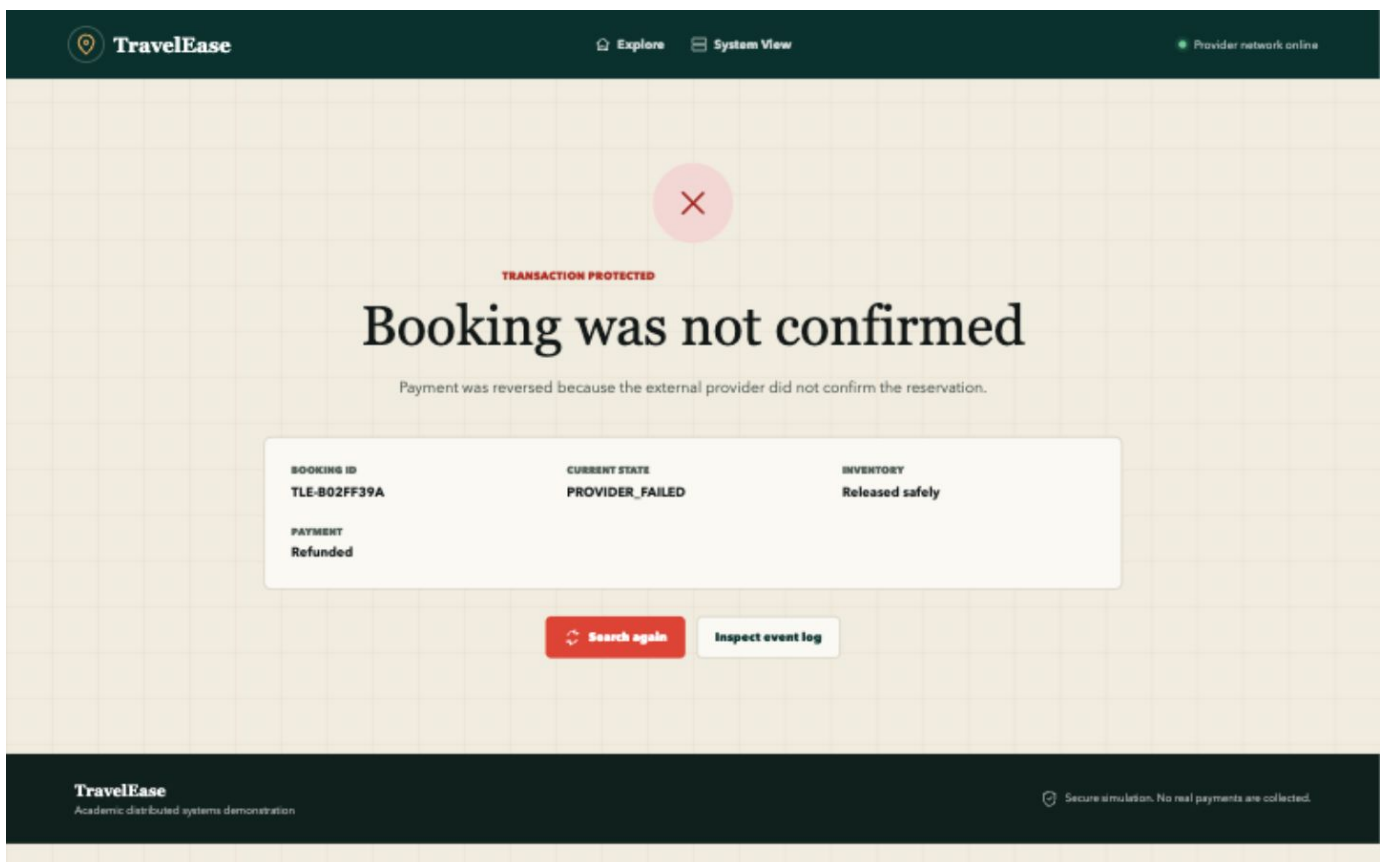


Figure 7. Provider timeout with compensating refund and inventory release

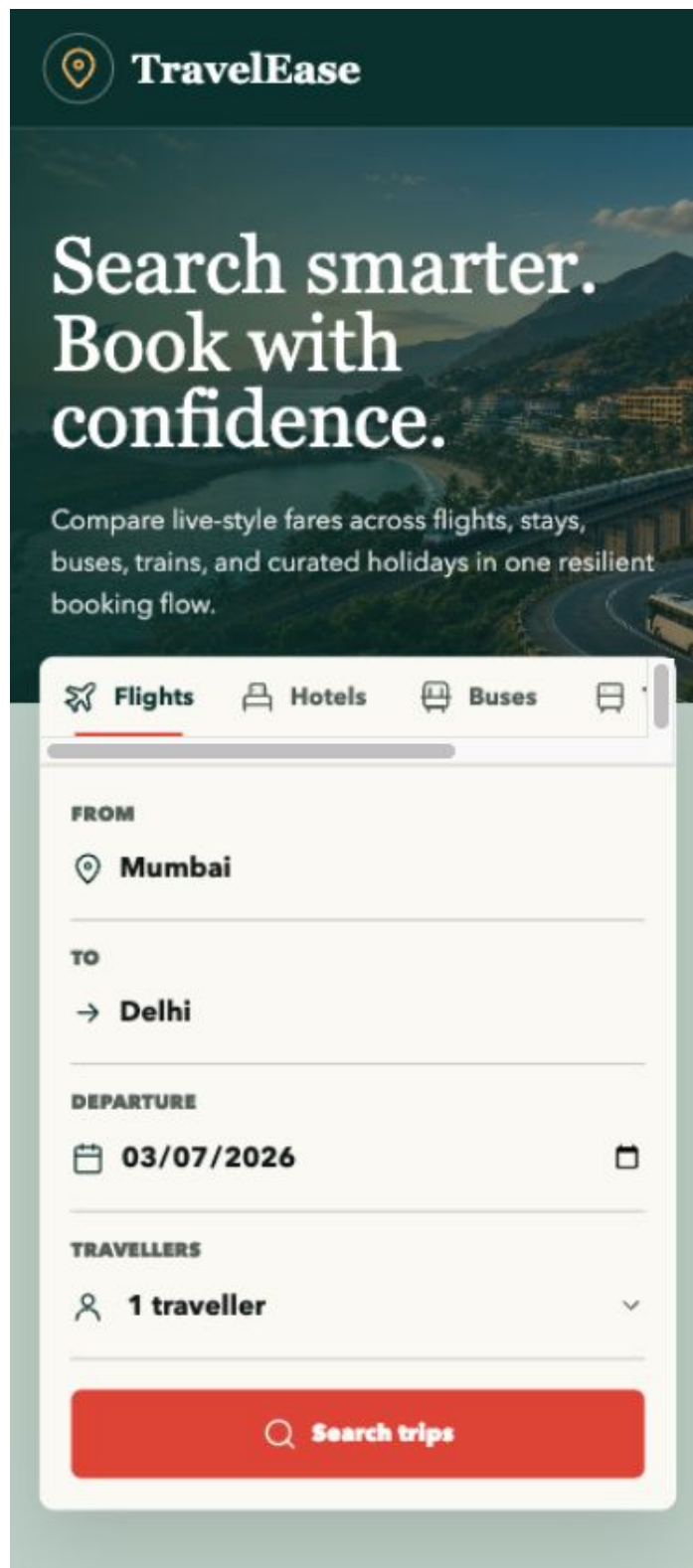


Figure 8. Responsive mobile search experience

Comparison with Real-World Systems

Large travel platforms use a similar separation between discovery and booking. Search systems rely on denormalized indexes, caches, ranking, and provider aggregation. The final reservation still requires authoritative supplier validation because displayed inventory and fares can change between search and payment.

Real systems additionally handle multiple currencies, fare rules, loyalty programs, fraud checks, tax jurisdictions, settlement, supplier contracts, asynchronous ticketing, PNR management, customer support tooling, and regulatory obligations. TravelEase focuses on the distributed-system principles behind those capabilities.

Future Scope

- Add authentication, traveller profiles, saved searches, and role-based operations access.
- Replace provider simulators with sandbox airline, hotel, train, bus, and payment APIs.
- Add quote-expiry countdowns and asynchronous pending confirmation.
- Add coupons, multi-currency pricing, taxes, refunds, and cancellation policies.
- Add a Redis cache and OpenSearch index through containerized local services.
- Add Kafka with transactional outbox and idempotent event consumers.
- Add recommendation models using browsing and booking events.
- Add fraud detection, risk scoring, and payment gateway failover.
- Add administrative dashboards, reconciliation tooling, and service-level monitoring.
- Deploy with containers, infrastructure as code, automated tests, and CI/CD.

Conclusion

TravelEase balances low-latency discovery with strongly consistent booking. The design scales search independently, protects inventory with short transactions, prevents duplicate payment through idempotency, and handles partial failures through Saga compensation and event-driven processing. The Flask project demonstrates these decisions in a form that can be executed, tested, inspected, and documented locally.